ISRG Internet Security Research Group

Memory Safety

Josh Aas

April 2026

Introduction

Internet Security Research Group (ISRG) is a nonprofit organization, founded in 2013 to serve as a home for public-benefit digital infrastructure projects.

ISRG is the organization behind Let's Encrypt, the free, automated, and open certificate authority serving 600M+ websites.

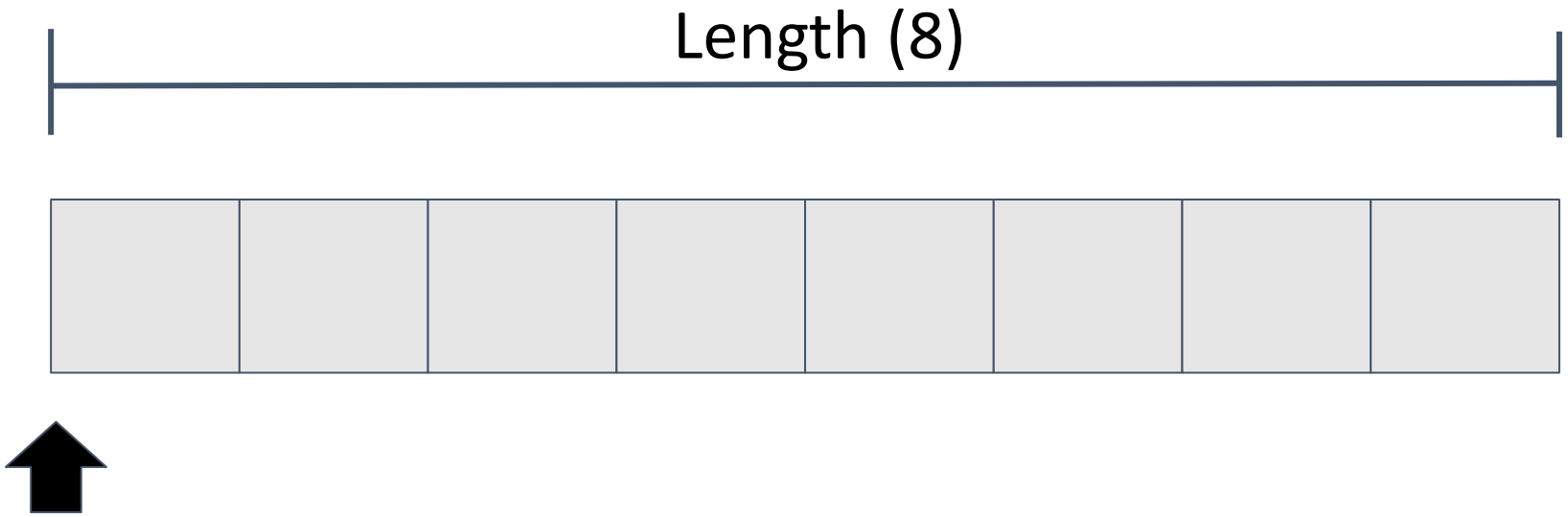
ISRG also runs Prossimo, a project to make the Internet's core software infrastructure memory safe.

Memory management overview

Program requests chunks of memory in which to store information. Called “memory allocation”.

Programs free memory when done with it so it can be allocated for something else.

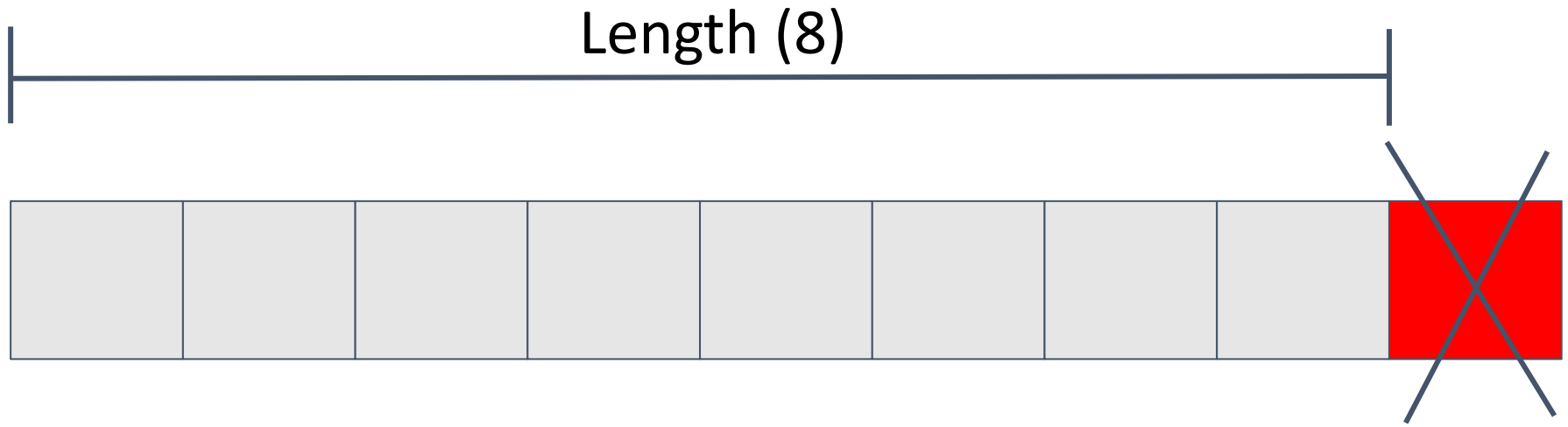
Memory Allocation Example



47384732

Starting address of allocated memory, each box is a byte

Memory management overview



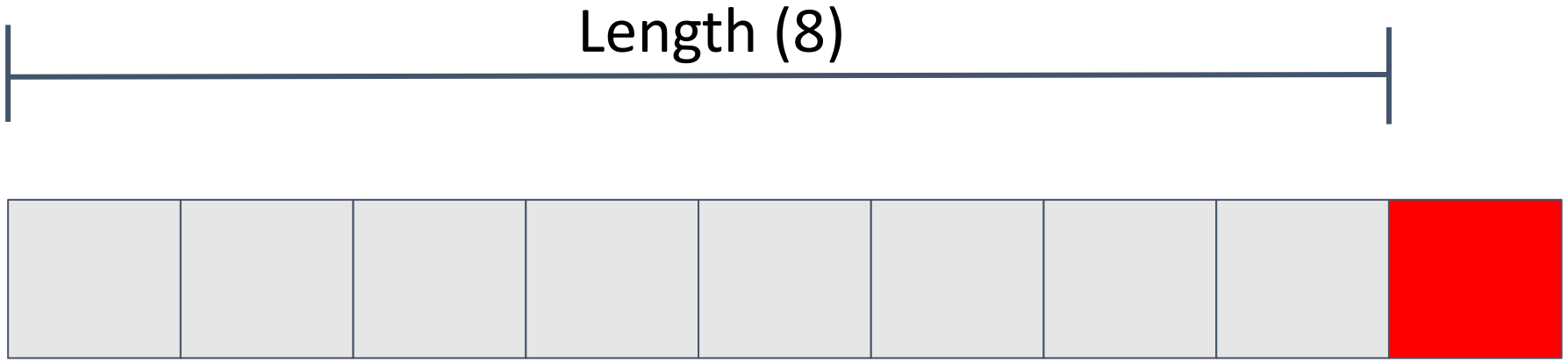
Program should only read and write within that chunk of memory, but that's not always what happens!

What can go wrong?

Exploitable issues:

- Out of bounds write (buffer overflow)
- Out of bounds read
- Use after free

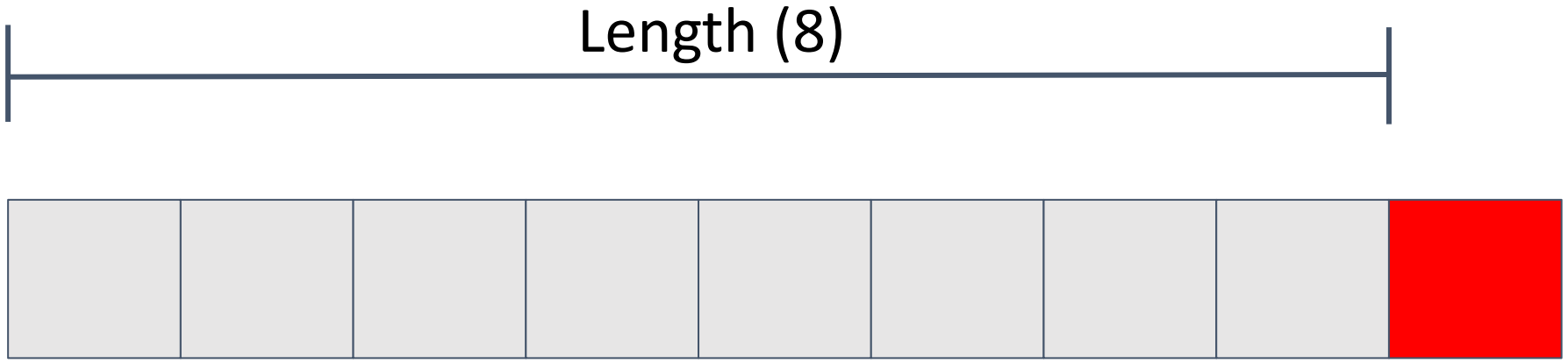
Out of bounds write



Writing 9 bytes -> **buffer overflow** or **out of bounds write**.

Overwrote memory for something else.

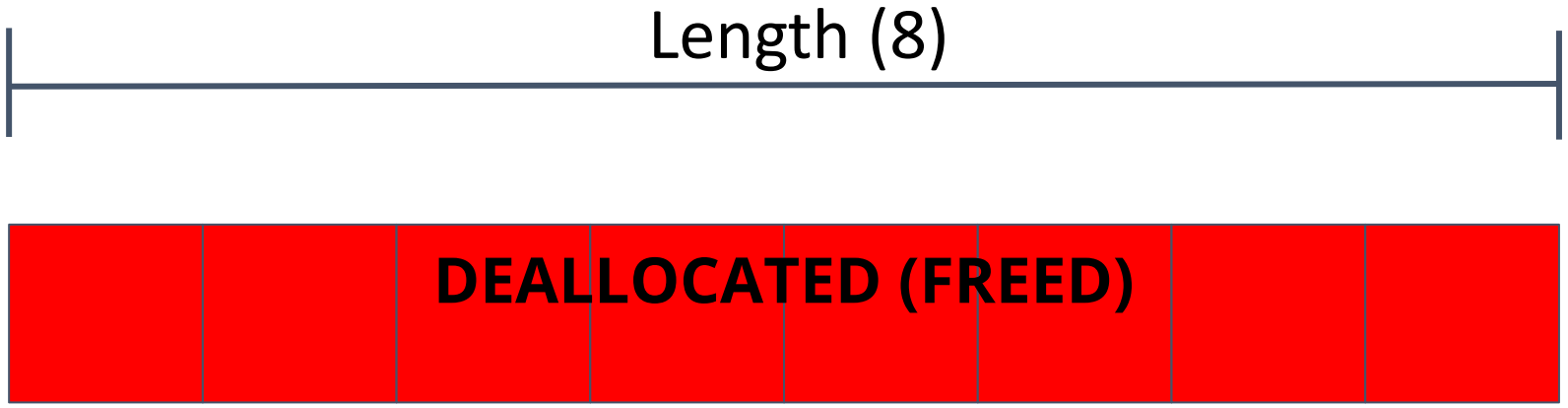
Out of bounds read



Reading 9 bytes -> **buffer overread** or **out of bounds read**.

Read memory that wasn’t supposed to be read.

Use after free



Reading memory after saying you're not using it any more.

That memory is now potentially allocated for something else.

Not all languages allow this!

C: Danger

C++: Danger

Java: Safe

Python: Safe

Rust: Safe

Swift: Safe

Unfortunately, a lot of important code is written in C and C++.

Safety vs. Performance

Used to be that there was a tradeoff between safety and performance. That's a primary reason so much important code is written in C and C++.

Not true any more, particularly with the advent of Rust.

How does this impact you?

Read the security release notes for software updates, look for the following words:

- Memory management
- Buffer overflow
- Out of bounds read/write
- Use after free

iOS 18.6.2 and iPadOS 18.6.2

Released August 20, 2025

ImageIO

Available for: iPhone XS and later, iPad Pro 13-inch, iPad Pro 12.9-inch 3rd generation and later, iPad Pro 11-inch 1st generation and later, iPad Air 3rd generation and later, iPad 7th generation and later, and iPad mini 5th generation and later

Impact: Processing a malicious image file may result in memory corruption. Apple is aware of a report that this issue may have been exploited in an extremely sophisticated attack against specific targeted individuals.

Description: An out-of-bounds write issue was addressed with improved bounds checking.

CVE-2025-43300: Apple

Numerous serious exploits

As of 2019, 70% of all security vulnerabilities in Microsoft products were memory safety vulnerabilities.

Android reported a similar percentage.

As best we can tell, Apple was in the same range.

Widespread Impact

Memory safety vulnerabilities affect society at all levels.

e.g. “WannaCry cyber attack cost the NHS £92m as 19,000 appointments cancelled”

e.g. “NSO Group’s iPhone Zero-Days used against a UAE Human Rights Defender”

Things are getting better!

“Microsoft is also committed to adopting memory-safe languages, such as Rust, for developing new products and transitioning existing ones.”

- Microsoft’s Secure by Design journey: One year of success, April 2025

Things are getting better!

"… the percentage of memory safety vulnerabilities in Android dropped from 76% to 24% over 6 years as development shifted to memory safe languages."

- Google Security Blog, September 2024

Things are getting better!

Apple recently published a big memory safety update:

Memory Integrity Enforcement: A complete vision for memory safety in Apple devices (September 2025)

Have been investing in memory safe Swift language for many years now.

What can you do?

1. Apply software updates promptly
2. Choose memory safe software if you have a choice
3. If you’re writing code, choose a memory safe language if possible (avoid C or C++)